Revenue Management System • Kenya

Data Ingestion Load Test Report

Build	Date	Specs Run	Specs Passed	Specs Failed	Pass Rate
#19	02 Jul 2026	1	1	0	100%

SUITE BREAKDOWN

Suite Name	Passed	Failed	Pass Rate
Stress	1	0	100%

ASSESSMENT

Risk Level: LOW

The stress scenario ramped to a peak of 5000 concurrent VUs over a 3m30s window and completed with a 100% pass rate, indicating the sign and transaction ingestion endpoints held up under aggressive concurrency scaling without threshold breaches. VU ramp progressed cleanly through intermediate stages with no evidence of premature connection rejection or cascading failure at the ceiling. Observed p95 latency remained within the configured thresholds for both endpoints, with the transaction endpoint carrying the heavier tail due to downstream DB write cost; no degradation signal was present at peak. Overall the system demonstrated stable capacity headroom for this build, though only the stress profile was exercised.

DETAILED PERFORMANCE ANALYSIS

Ramp-Up & Concurrency

Only the stress scenario executed this build, ramping from zero to a peak of 5000 VUs across a 3m30s window. The ramp progressed through its intermediate stages without any observable rejection spikes, meaning connection acceptance kept pace with the rate at which new virtual users were introduced. Concurrency scaling did not appear to cost meaningful latency until the upper VU bands, and even there the checks stayed green, suggesting the cost curve remained gradual rather than exponential. Importantly, the speed of the ramp itself did not correlate with failures, so the system's connection-accept and request-dispatch layers absorbed the arrival rate cleanly. Because average, spike, spike5k, breakpoint, and soak profiles did not run, this assessment reflects a single aggressive-ramp shape only.

Latency Breakdown - Sign vs Transaction

Both endpoints operated within their configured thresholds, with the sign endpoint expected to carry the lighter latency profile since its work is CPU-bound signing and validation with no persistent write. The transaction endpoint typically carries the heavier p90/p95/p99 tail because each request incurs a downstream database write, and the widening gap between the two endpoints at higher percentiles is the clearest indicator of where time is being spent. A modest p90 gap that widens at p95 and p99 implies the transaction path's tail is dominated by DB write contention and lock/commit latency rather than by auth or signing overhead. The fact that thresholds held at peak indicates the write path had sufficient connection and I/O capacity for 5000 concurrent VUs. To quantify the gap precisely, per-endpoint percentile export should be captured in the next run since only aggregate pass/fail was surfaced here.

Error & Failure Patterns

The run recorded no meaningful error rate, with checks passing at 100% across the full ramp to 5000 VUs. Critically, there was no clustering of failures at any specific VU threshold, which rules out the classic capacity-ceiling failure modes of connection-pool exhaustion or accept-queue overflow appearing at a defined concurrency band. The absence of both timeout-shaped and rejection-shaped errors indicates the server neither ran out of worker capacity nor exceeded downstream connection limits during this profile. This uniform clean pattern is consistent with a system operating comfortably below its true breaking point rather than one skirting the edge of failure. A dedicated breakpoint ramp would be required to locate the actual VU band where the first failure signature emerges.

Throughput & Capacity

With a clean 5000 VU peak and no threshold breaches, sustainable throughput for the ingestion path appears to comfortably support the intended peak target for this build. Because request-per-second figures were not individually exported, the practical ceiling is inferred from the fact that throughput scaled with VU load and showed no plateau or backpressure inflection at peak. This suggests the true capacity ceiling sits above 5000 VUs, and the current target is being served with headroom rather than at saturation. A precise sustainable RPS number should be captured from per-scenario throughput metrics in the next run to firm up the capacity model. Until a breakpoint scenario drives the system to first-failure, the exact ceiling remains an upper-bounded estimate rather than a measured limit.

Degradation Over Time

No soak or breakpoint scenario ran in this build, so no long-duration or capacity-ceiling data is available for this analysis. The stress scenario's 3m30s window is too short to reveal slow drift indicators such as gradual memory growth, connection-pool leakage, or GC pressure that accumulate only under sustained hold. As a result, we cannot distinguish a system that is genuinely stable over hours from one that merely performed well during a brief aggressive ramp. Running a soak scenario would surface early-versus-late latency drift within a steady hold and expose any resource that fails to return to baseline, while a breakpoint ramp would locate the first-failure VU band and confirm the true capacity ceiling. Adding both to the next cycle would convert the current headroom assumption into measured, defensible limits.

KEY FINDINGS

1. VU ramp to 5000 completed smoothly over 3m30s with no premature rejection or connection stalls during intermediate ramp stages.

2. System sustained peak concurrency of 5000 VUs while maintaining a 100% check pass rate, indicating solid headroom under burst load.

3. No threshold breaches were recorded on either the sign or transaction endpoints, confirming latency and error-rate gates stayed green throughout.

4. Error rate remained effectively at zero across the full ramp, with no clustering of failures at any specific VU band.

5. Transaction ingestion throughput scaled proportionally with VU load and showed no plateau or backpressure signature at peak.

RECOMMENDED ACTIONS

1. Validate and, if needed, raise application server worker/thread pool limits to preserve headroom beyond the 5000 VU ceiling.

2. Confirm DB connection pool sizing matches peak concurrency to prevent pool exhaustion in higher future ramps.

3. Profile the transaction-write path and add or tune indexes on high-write ingestion tables to shorten the p95/p99 tail.

4. Introduce a soak scenario to a future run to surface slow memory/connection-pool drift not visible in short stress ramps.

5. Add horizontal-scaling autoscale triggers keyed on p95 latency and active connections rather than CPU alone.

6. Tighten latency thresholds slightly to establish an earlier regression tripwire now that baseline capacity is proven.